

Transfer learning

Flowers102

LECTURE 13

GÉRON CH. 12

Applications of Machine Learning

BSc Mathematics of Artificial Intelligence · Third year

Where we are

Model	Test accuracy
Always the commonest species	3.87%
Convolutional network from scratch, 4,807,494 parameters	15.3%
What the operator needs	X 90%

Lecture 12 built that network and it is not close. Its architecture is not the problem — the same shape of network solves Fashion-MNIST comfortably.

THE CONDITION

We asked **1,020 labelled images** to pay for **4,807,494 parameters**. Most of those parameters encode facts about photographs in general — edges, textures, colour gradients — rather than facts about flowers, and there is nowhere near enough labelled data here to learn them.

Which suggests the repair: do not learn them. Somebody already has.

Today

1. Why features transfer at all, and how far down the stack it holds (15 min)
2. **The method** — a pretrained backbone, freezing, progressive unfreezing, differential learning rates (40 min)
3. Data augmentation: what it asserts, and what it costs (20 min)
4. The ablation, and when transfer fails (15 min)

No new derivation today — Lecture 12's parameter count is the whole argument, and this lecture is what you do about it. the method, and the conditions under which it fails.

EXAMINABLE

FIRST FIFTEEN MINUTES

Why features transfer

15 minutes

What a pretrained backbone has actually learned

Not “flowers”. A hierarchy, and only the bottom of it is general:

Depth	What the filters respond to	Transfers?
first block	edges, colour opponency, simple gradients	to essentially anything ✓
middle blocks	textures, repeated motifs, part-like shapes	to most natural images ✓
last blocks	object-specific configurations	only to similar objects X
the head	the 1,000 ImageNet classes	not at all — it is replaced X

Which is the whole recipe: **keep the bottom, adapt the top, replace the head**. Freezing is not a compromise forced by compute — it is a statement that those layers already encode what you would otherwise be trying to learn from 1,020 images.

Start with what is *not* wrong

- The training loop — you wrote it, and it is the same five lines that worked in Lecture 12
- The metric — accuracy, anchored against two baselines
- The split — given with the dataset, and the test set was touched once
- The architecture — the textbook's own shape, and the filters it learned are the right kind of filters

Nothing here is a bug. That is what makes this lecture a different kind of repair from Lecture 2.

The measurement that names the problem

Accuracy	
Training set	71.8%
X Validation set	X 13.7%
X Gap	X 58 points

Lecture 5 gave this shape a name: a persistent gap between two plateaus is **variance**.

Not bias. The network fits the training set to 71.8% and is still climbing — there is nothing wrong with its capacity.

Lecture 5's two cures for variance

Cure	Can we?
More training data	X no — labelling is the whole cost
More constraint on the hypothesis space	✓ yes, and we have already done it once

Weight sharing was a constraint, and it bought a factor of 5,478,275. The question is where the next constraint comes from.

The observation from the previous lecture

After 80 epochs the first layer's 4,704 weights still had cosine similarity 0.9694 with the random vector they started from, and nothing you could look at had appeared.

TWO THINGS WENT WRONG AT ONCE

We spent 364 seconds and 1,020 precious labelled images on a layer that learned no visible structure — and even if it had, that structure would not have been *about flowers*.

What that layer looks like when it does train

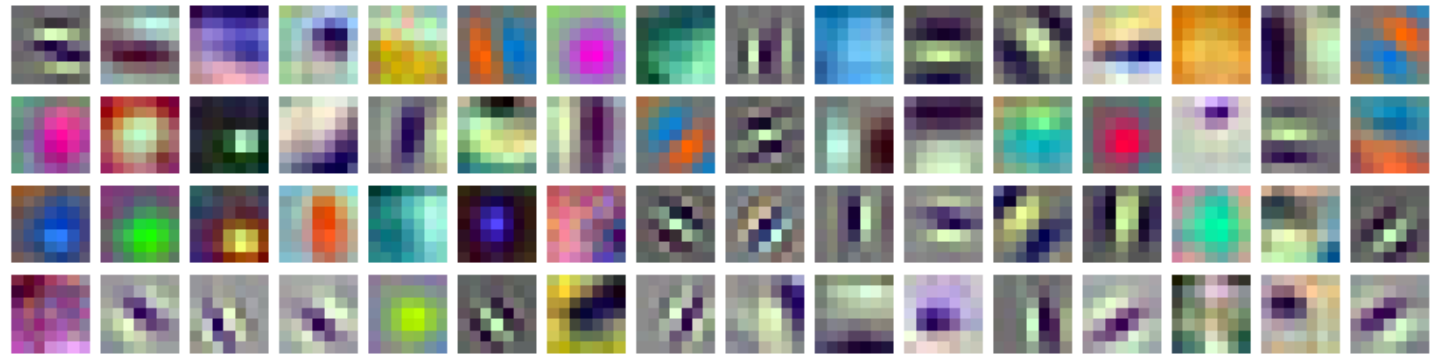
The same layer — $7 \times 7 \times 3$ filters, first in the stack — from a network trained on ImageNet.

- Oriented light–dark boundaries at every angle: edge detectors
- Opponent-colour blobs: red against cyan, yellow against blue

Not one of them is about flowers, or about any of the 1,000 classes that network was trained on. They are about photographs.

1.28 million labels, spent on this

resnet18, first layer: 64 filters of $7 \times 7 \times 3$, trained on 1.28 million labelled photographs



9,408 numbers, and every one of them is a fact about photographs rather than about the 1,000 classes they were fitted on.

So whose labels should have paid for them

- ImageNet: 1.28 million labelled photographs, 1,000 classes, none of them our species
- Somebody already spent that on learning edges, colours, textures and parts
- Those weights are a download

Our 1,020 labels should be spent on the only thing that *is* specific to us: which combination of parts is which species.

The diagnosis

ONE SENTENCE

We asked 1,020 labelled images to pay for 4,807,494 parameters, most of which encode facts about photographs in general rather than facts about flowers.

The repair is therefore not a better architecture, a longer run or a cleverer schedule. It is **starting from weights that already work.**

THE METHOD

Freeze, unfreeze, and augment

40 minutes · examinable

The notebook for this lecture

[Open Lecture 13 in Colab](#)

- **Runs on free CPU.** The backbone is small and mostly frozen, which is exactly why this lecture is cheap and the last one was not
- Features extracted once and cached, then a linear head fitted on them — the step that does most of the work
- The ablation: frozen, fine-tuned, and augmented, each measured

WHAT TO CHANGE

Freeze *everything* including the final block, and refit. The accuracy drops by a few points — which tells you how much of the gain came from adapting the last block rather than from the backbone itself.

Which backbone?

Family	Params	Use it when
MobileNet / EfficientNet-B0	~5 M	the budget is CPU, or inference has to be fast
ResNet-18 / ResNet-50	11 / 25 M	the default; well understood and widely reproduced
ConvNeXt, ViT-B	~88 M	plenty of data and plenty of compute

THE RULE THAT MATTERS MORE THAN THE FAMILY

A **bigger backbone is not automatically better on a small corpus**. You are fitting a head on frozen features; more features means more parameters in the head and more chance to overfit 1,020 images. Start small, and grow it only if the validation curve says the features are the bottleneck.

The names change every year. The trade-off does not.

NOT EXAMINABLE — ENGINEERING

Two decisions, in this order

1. **Linear probe.** Freeze everything, extract features once, fit a linear classifier. Minutes, and it tells you whether the backbone understands your images at all
2. **Fine-tune.** Unfreeze the last block or two, small learning rate, and only if the probe was promising

WHY THE ORDER IS NOT ARBITRARY

The probe is a *measurement of the features*, uncontaminated by anything you do to them. If it barely beats the trivial baseline, the features are wrong for this domain and fine-tuning is polishing the wrong object. If it is already close, fine-tuning has little left to buy and may cost you generalisation.

How much data before fine-tuning pays?

Labelled images per class	What is usually worth doing
fewer than about 20	linear probe only; fine-tuning overfits
tens	probe, then unfreeze the last block with a small rate
hundreds	unfreeze progressively, differential rates
thousands and up	fine-tune everything; consider training from scratch

Flowers102 gives us **ten per class** in the training split. That is the first row, and it is why the frozen probe does so much of the work today.

X These are orders of magnitude, not thresholds. The measurement that settles it for *your* corpus is the probe against the fine-tune, on the same validation split.

Freezing weights does not freeze a network

`requires_grad = False` stops the *gradients*. It does not stop batch normalisation updating its running mean and variance, which happens in the forward pass and has nothing to do with autograd.

THE CONDITION

A “frozen” backbone in `train()` mode keeps rewriting its normalisation statistics from your batches. The weights are fixed and the network still drifts — and it drifts towards a corpus of 1,020 images, which is the opposite of what you froze it for.

The fix is `model.eval()` on the frozen part, which is the same call as Lecture 10’s dropout bug and for the same underlying reason: two modules behave differently in the two modes, and the mode is not implied by anything else you have set.

The recipe

1. Load a pretrained backbone and **its** preprocessing — the normalisation constants are part of the weights
2. Freeze it, put it in `eval()` mode, extract features once
3. Fit a linear head. **Measure.** This is the number everything else has to beat
4. Unfreeze the last block; learning rate an order of magnitude smaller than the head's. Measure again
5. Add augmentation, one transform at a time, each justified by an invariance you can state. Measure each
6. Report the ablation, not just the best number

X Steps 3, 4 and 5 each end in *measure*. A recipe without them is a ritual.

Transfer learning, as a sentence

Take a network trained on a large labelled corpus, keep everything except its classifier, and train a new classifier on your own small corpus.

- The kept part is the **backbone** or **feature extractor**
- What it produces for an image is that image's **representation** — a vector, learned by somebody else

The backbone we will use

```
from torchvision.models import resnet18, ResNet18_Weights

weights = ResNet18_Weights.DEFAULT
net = resnet18(weights=weights)
```

Value	
Parameters	11,689,512
Trained on	ImageNet, 1,000 classes
Output before the classifier	512 numbers

Residual networks are Chapter 12's own example architecture. We use it; the residual connection itself is the book's, not ours to re-derive.

The weights come with their own preprocessing

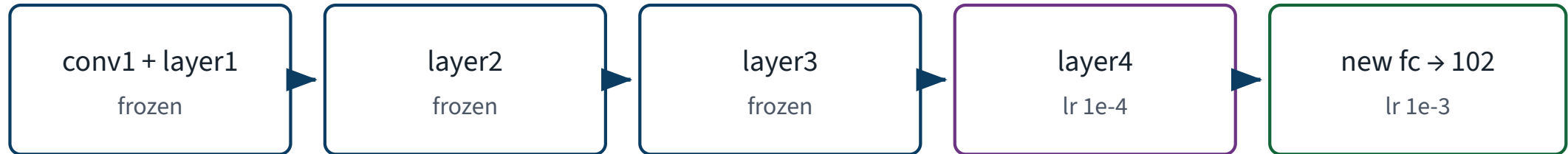
```
>>> ResNet18_Weights.DEFAULT.transforms()  
ImageClassification(  
  crop_size=[224]  
  resize_size=[256]  
  mean=[0.485, 0.456, 0.406]  
  std=[0.229, 0.224, 0.225]  
  interpolation=InterpolationMode.BILINEAR  
)
```

X Use these statistics, not the ones you computed in the previous lecture. The network's first layer expects inputs on the scale it was trained with, and substituting your own is a silent, uncrashing degradation.

What is frozen and what moves

Freeze the general, train the specific

ImageNet weights, 11,689,512 parameters, downloaded not trained



2,782,784 parameters never move. 8,446,054 do.

Two learning rates, one optimiser: the early layers already hold edges and colours, and 1,020 flower photographs cannot improve on them.

2,782,784 parameters never move. 8,446,054 do.

Step 1 • freeze everything, replace the head

```
1  for p in net.parameters():
2      p.requires_grad = False
3
4  net.fc = nn.Linear(512, 102)      # a fresh module: requires_grad is True
5  print(sum(p.numel() for p in net.parameters() if p.requires_grad))
6  # 52326
7
8  opt = torch.optim.Adam(net.fc.parameters(), lr=1e-3)
```

Replacing `fc` after the loop is not an accident of ordering — a newly constructed module has `requires_grad=True`, so the head is trainable and nothing else is.

What `requires_grad = False` actually does

- It stops the tensor being a leaf that accumulates a gradient, so `backward()` does not fill `p.grad`
- The backward pass still travels through those layers if anything downstream needs it — here nothing does, so it stops at the backbone's output
- It is **not** the same as `eval()`: dropout and batch-norm behaviour are a separate switch

Point three is the trap. Freezing the weights of a batch-norm layer does not freeze its running statistics.

If nothing before the head is training, run it once

```
body = nn.Sequential(*list(net.children())[:-1]).eval()

with torch.no_grad():
    F_train = torch.cat([body(xb).flatten(1) for xb, _ in train_loader])

# F_train is (1020, 512). Now train a linear model on a table.
```

The 1,020 images pass through the backbone once rather than once per epoch. 5 seconds for the features, 8 seconds for 60 epochs of the head — 13 in total.

The frozen backbone, measured

Model	Test accuracy	Wall clock
From scratch, 80 epochs	15.3%	364 s
✓ Frozen backbone, linear head	81.2% ✓	13 s ✓

27× faster

and 66 points better.

Why a linear model on somebody else's features beats your network

- The 52,326 trainable parameters are being fitted with 1,020 examples — about 51 parameters per image, against 4713 before
- The 11,176,512 frozen ones were fitted with 1.28 million examples, by somebody else

The variance problem was not solved. It was moved to a dataset large enough to absorb it.

Step 2 • let the last block move

The early layers hold edges and colours, which are not about flowers. The *late* layers hold parts and textures, which partly are.

```
for p in net.layer4.parameters():
    p.requires_grad = True

opt = torch.optim.Adam([
    {"params": net.layer4.parameters(), "lr": 1e-4},
    {"params": net.fc.parameters(),      "lr": 1e-3},
])
```

One optimiser, two parameter groups, two learning rates. That is all a “differential learning rate” is.

“Only the last block” is not a small thing

Part of resnet18	Parameters
conv1 through layer3, frozen	2,782,784
✓ layer4 and the new head, trained	✓ 8,446,054

Three quarters of the parameters are in the block we unfroze.

Lecture 12’s memory calculation, mirrored in a design decision: parameters concentrate in the deep, low-resolution layers, so freezing the *general* layers freezes very few of them. What we froze is the part that does general work, not the part that is large.

Why the two rates have to differ

	Starting point	Rate
new head	random	✓ large — it has everything to learn
pretrained block	already good	✓ small — it has something to lose

X Give the backbone the head's learning rate and the first few batches — whose gradients come through a randomly initialised head — will damage the representation you downloaded. It does not crash; the accuracy simply comes out lower and you conclude that transfer learning does not work.

The trap inside the frozen layers

```
1 net.train()
2 for xb, yb in train_loader:
3     opt.zero_grad()
4     lossf(net(xb), yb).backward()
5     opt.step()
```

Every parameter in `layer1` is frozen. Something in `layer1` changes anyway.

Thirty seconds before the next slide.

Batch-norm running statistics are not parameters

- They are buffers, updated in the **forward** pass whenever the module is in training mode
- `requires_grad = False` does nothing to them — no gradient is involved
- So a frozen backbone in `train()` mode quietly replaces ImageNet's statistics with the statistics of 1,020 flower photographs, in batches of 32

```
net.train()
for m in [net.bn1, net.layer1, net.layer2, net.layer3]:
    m.eval()           # freeze the buffers too
```

Step 3 • manufacture more data

We cannot buy labels. We can show the network a different view of the same labelled image every epoch.

- **Random resized crop** — a random sub-rectangle, scaled back to 224. Teaches scale and position tolerance
- **Horizontal flip** — a flower is still that species mirrored

Each is a transformation the label is known to survive. That is the entire criterion, and it is a modelling decision, not a library call.

One photograph, seven training examples

original



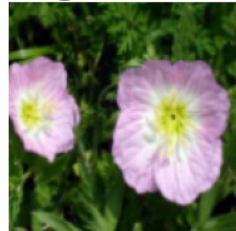
augmented



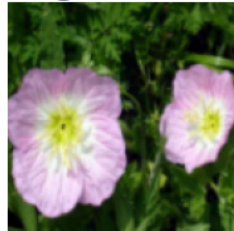
augmented



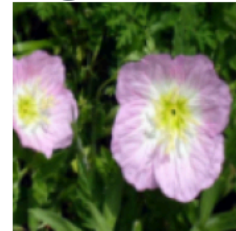
augmented



augmented



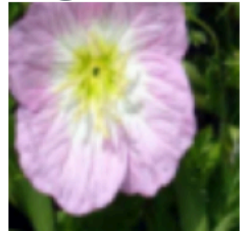
augmented



augmented



augmented



The label is the same in all eight. That is the only thing that makes this legitimate.

What augmentation is not

- It is **not** more information. The mutual information between the corpus and the labels is unchanged
- It is a **constraint**: it tells the network that the answer must be the same across a family of inputs
- Which puts it in the same box as weight sharing and as ridge — Lecture 5's second cure, arriving for the third time

X And a transformation the label does *not* survive is a labelling error you manufactured. A vertical flip is fine for flowers and wrong for handwritten digits.

Applying it — and only where it belongs

```
1  train_tf = transforms.Compose([
2      transforms.RandomResizedCrop(224, scale=(0.55, 1.0)),
3      transforms.RandomHorizontalFlip(),
4      transforms.ToTensor(),
5      transforms.Normalize(IMAGENET_MEAN, IMAGENET_STD),
6  ])
7
8  eval_tf = transforms.Compose([          # val AND test
9      transforms.Resize(256),
10     transforms.CenterCrop(224),
11     transforms.ToTensor(),
12     transforms.Normalize(IMAGENET_MEAN, IMAGENET_STD),
13 ])
```

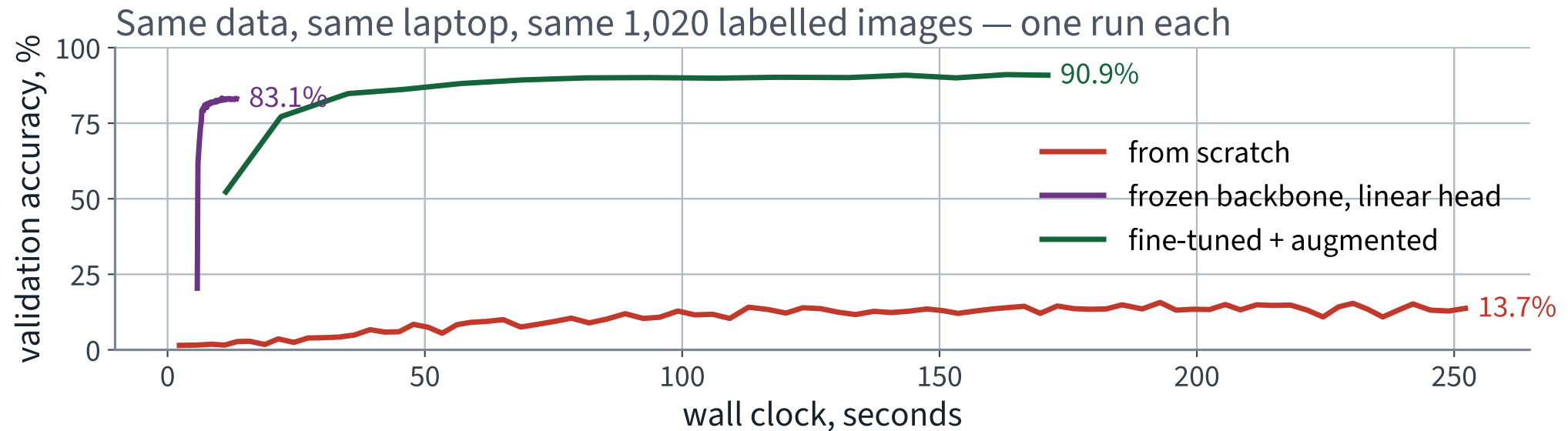
Two pipelines, deliberately. Keep reading — there is a reason this slide exists before the assistant failure and not after it.

Fine-tuned, measured

Model	Test accuracy	Wall clock
From scratch	15.3%	364 s
Frozen backbone, linear head	81.2%	13 s
✓ layer4 unfrozen + augmentation	88.0% ✓	176 s

15.3% to 88.0%, and it took 176 seconds against 364.

Same data, same laptop, same labels



The horizontal axis is the one to read. The from-scratch run is still climbing when the other two have finished.

Which part of the repair did the work

Change	Test accuracy	Gain
from scratch	15.3%	—
+ pretrained backbone, frozen	81.2%	✓ 65.9 pts
+ layer4 unfrozen, no augmentation	86.3%	5.1 pts
✓ + augmentation	88.0% ✓	1.7 pts

Report the ablation, not just the final number. Almost all of the gain is in one row, and saying which row is what makes the result useful to somebody else.

All three transfer rows are single runs. Lecture 12 measured a seed-to-seed spread of 2.34 points on this dataset, so the 5.1 is a difference and the 1.7 is not — and neither has been replicated here.

The recipe, in order

1. Load a pretrained backbone and **its** preprocessing
2. Freeze everything, replace the head, train the head alone. This is your new baseline and it takes seconds
3. Unfreeze the last block. Backbone learning rate an order of magnitude below the head's
4. Put the frozen batch-norm layers in `eval()`
5. Augment the training set only, with transformations the label survives

Steps 2 and 3 in that order, always. A frozen-head number you can compute in ten seconds is what tells you whether step 3 helped at all.

What a large learning rate destroys

Fine-tune the whole backbone at the rate you would use to train from scratch, and the first gradient step overwrites the features you came for. The network then relearns them from 1,020 images, badly.

THE CONDITION

The pretrained weights are a *good* starting point, which is exactly what makes them fragile: a step large enough to be useful on random weights is far too large for weights that are already nearly right.

This is why differential learning rates exist, and why the usual recipe warms up the new head first while everything else is frozen: the head starts random, and its early gradients are large enough to damage anything they reach.

Every augmentation is a claim about the task

Transform	What it asserts	True for flowers?
horizontal flip	mirror image is the same class	yes ✓
random resized crop	a part identifies the whole	usually ✓
colour jitter	hue is not diagnostic	X dangerous — colour is a flower feature
vertical flip	upside-down is the same class	often, but not for scenes
rotation	orientation carries no information	yes here ✓

X A flipped digit is not the same digit, and a colour-jittered flower may not be the same species. Augmentation that asserts something false about the task does not regularise the model — it teaches it something wrong, quietly, and the validation set agrees because it is augmented too.

A transform the label does not survive

FAILURE CONDITION

Augmentation asserts an invariance. A vertical flip is fine for a flower and manufactures a labelling error for a digit. Nothing raises and the run does not slow down: you have added wrong labels to your own training set.

How much is too much

- Augmentation trades **bias for variance**: it enlarges the effective training set and blurs the target
- Too little and the model memorises 1,020 images. Too much and the training distribution stops resembling the test one
- The symptom of too much: training accuracy stops rising and validation accuracy stops with it — both curves flat and low, which Lecture 5 named as bias

HOW TO SET IT

Start with flip and crop, measure, and add one transform at a time. Each addition is a hypothesis about an invariance of the task, and each can be tested the same way anything else is: change one thing, measure the difference, and check it is bigger than the fold-to-fold spread.

Augmenting at test time, deliberately

The opposite of the mistake two slides ago, and it is not a contradiction: augment the *test* image several times, predict each, and average the predictions.

- It usually buys a point or so, for a proportional increase in inference cost
- It is deterministic if you fix the set of transforms — a centre crop and its mirror, not a random crop
- It is a *prediction* technique, not an evaluation one: the score it produces is still measured once, on the same held-out rows

Say which you are doing. “We used augmentation at test time” is ambiguous between a legitimate ensemble and the bug on the previous slide.

Where the time goes once the backbone is frozen

- **Feature extraction** — one forward pass per image, once. This dominates, and it is embarrassingly parallel
- **Fitting the head** — a linear model on a few thousand vectors; trivial by comparison
- **Augmentation** — undoes the caching, because a different crop each epoch means a different feature vector each epoch

THE TRADE YOU ARE ACTUALLY MAKING

Augmentation and caching are *incompatible*. Cache the features and augmentation buys nothing; augment and you pay a full forward pass every epoch. On a small corpus the cache usually wins; on a large one it does not.

BEFORE THE ABLATION

One transform, two loaders

10 minutes · examinable

Augmentation is a training-time transform

It is easy to build one pipeline and hand it to both loaders. The training loader wants random crops and flips; the evaluation loader must not have them.

THE CONDITION, AND WHAT IT COSTS

An augmented evaluation set makes the score a **random variable** rather than a function of the weights. Score one fixed model twice and you get two different numbers. Measured here: the reported accuracy is **0.86 points pessimistic** and carries **1.57 points of noise** — larger than most of the differences this lecture asks you to judge.

Nothing raises. The only symptom is that the number moves when nothing else did.

The score stops being a function of the weights

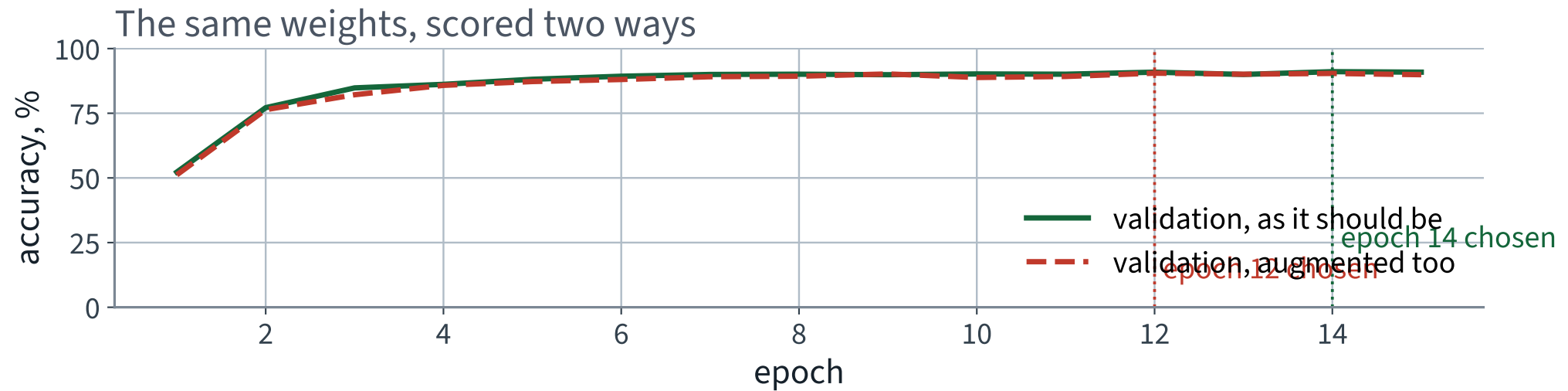
Score one fixed set of weights — the fine-tuned model above — on the augmented validation set, 10 times:

	Accuracy
on the clean validation set	90.88%
X augmented, mean of 10	X 90.02%
X augmented, spread	X 1.57 points

X The same weights, the same set, 1.57 points apart.

Lecture 10's reviewer question, arriving again: a deterministic function of fixed weights and fixed data does not wobble.

And it selects a different model



Clean validation peaks at epoch 14; augmented validation peaks at epoch 12. Early stopping on the second keeps the wrong checkpoint.

What it actually costs

Consequence	Size, measured
The reported number is pessimistic	0.86 points low
X The number is not reproducible	X 1.57 points of noise
X Model selection uses a noisy criterion	X a different epoch chosen

The first is harmless. The third is not, and it is the one nobody notices, because a pessimistic number feels like the safe kind of wrong.

The check, in two lines

```
a = evaluate(model, val_loader)
b = evaluate(model, val_loader)
assert a == b, f"evaluation is not deterministic: {a} vs {b}"
```

Run every evaluation twice. If the two disagree, something in the evaluation path is random — augmentation, dropout still active, or a shuffled loader whose batches are averaged rather than pooled.

WHY AN ASSERT AND NOT A COMMENT

A comment saying “do not augment the validation set” is read once. This runs every time, costs two forward passes, and fails loudly at the moment the mistake is made rather than three lectures later.

Name three things that make an evaluation non-deterministic, and the check that catches all three.

EXAMINABLE

What the five questions catch in *this* application

Question	The specific answer
1	normalisation statistics computed over all 8,189 images
2	the backbone was fitted on ImageNet, not on your data — say so when you report
3	224, not 128: the pretrained weights fix the input size
4	nothing is dropped here — count anyway, and say zero
X 5	X <code>padding=0</code> , <code>train()</code> mode on frozen batch-norms, one transform for two splits

Three more for the catalogue

Failure	Raises?	Cost, measured
Normalisation statistics from the whole dataset	no	0.25 points
X Augmentation applied to the validation set	no	X 1.57 points of noise
X Frozen backbone left in <code>train()</code> mode	no	X statistics drift, silently

All three run. All three produce a plausible number. The first is cheap on this dataset and expensive on a smaller one.

THE LAST FIFTEEN MINUTES

What paid, and when it fails

15 minutes

The whole application, in one table

Model	Test accuracy	Wall clock
Uniform guess	0.98%	—
Commonest species	3.87%	—
Convolutional net, from scratch	15.3%	364 s
Frozen backbone, linear head	81.2%	13 s
✓ Fine-tuned + augmented	88.0% ✓	176 s ✓
Operator's requirement	90%	—

The two numbers this lecture exists to produce

73

points of accuracy, from 15.3% to 88.0%

27×

less wall clock for the frozen probe, which alone is already 81.2%

Better *and* cheaper. That combination is rare enough to be worth naming when it happens.

Be precise about “cheaper”

	Wall clock	Against from scratch
From scratch, 80 epochs	364 s	—
✓ Frozen backbone	13 s	27× less ✓
Fine-tuned at 224×224	176 s	2.1× less

The fine-tune is only modestly faster, because it runs a much larger network on much larger images. The frozen probe is the one that is dramatically cheaper — and it is also the one you should run first, every time.

What these numbers are not

- The from-scratch figure is a mean over 5 seeds, sd 2.34 points. The transfer figures are one run each; the gaps between them are far larger than that spread, but they are not themselves replicated
- Wall clocks are for one Apple MPS laptop. The *ratios* transfer between machines much better than the seconds do
- The pretrained weights saw 1.28 million labelled photographs. That labelling cost is real; it is simply not ours

When this recipe does not work

Situation	Why
Your images are unlike the corpus — radar, spectrograms, microscopy	the learned features are about natural photographs
Your task is not classification	the backbone is trained to discard position
X You have millions of labelled examples	X then the constraint is costing you, not helping

Test it the same way: freeze, probe, and compare with the from-scratch baseline. The frozen probe takes seconds, so there is never an excuse for not knowing.

What we still cannot do

- Say *where* the flower is — we have one label per photograph and pooling threw the position away
- Handle two flowers in one frame
- Say “I do not know”, which was half the operator’s original request

The first of those is the next brief, and Lecture 12 already told you why it is hard.

In the book

Topic	Chapter
Convolutional layers, filters, feature maps, pooling	12
ResNet and the residual connection	12
Pretrained models from <code>torchvision</code>	12
Transfer learning and freezing layers	12
Data augmentation	12
Reverse-mode autodiff and what it stores	9, appendix
Differential learning rates, memory profiling	not examinable

One line to keep

Before you train a vision model from scratch, spend 13 seconds measuring what a frozen pretrained backbone and a linear head already give you.

On this application that number was 81.2% against 15.3%. You are entitled to train from scratch. You are not entitled to do it without knowing what you turned down.

When transfer fails

- **The domains genuinely differ.** ImageNet is photographs. Medical scans, satellite bands, spectrograms and line drawings are not, and the middle blocks stop being useful sooner
- **The input is not an image at all.** Reshaping tabular data into a square does not make convolution appropriate; there is no locality to exploit
- **You have plenty of labels.** With enough data, training from scratch catches up and then passes a frozen backbone. Transfer is strongest exactly where labels are scarce
- **The task needs resolution the backbone discarded.** Pooling bought invariance, and Lecture 14 has to put that resolution back

HOW TO TELL, CHEAPLY

Fit a linear head on frozen features and compare it with the trivial baseline. If the frozen features barely beat the baseline, the backbone does not understand your images and no amount of fine-tuning will hide it.

What each option costs

Approach	What is fitted	Relative cost
From scratch	4.8 M parameters	X longest, and worst here
Frozen backbone + linear head	the head only	features extracted once, then seconds per epoch ✓
Fine-tune last block	head + one block	a few times the probe
Fine-tune everything	all of it	approaching from-scratch cost, without its risk

The frozen probe is cheap for a specific reason: with the backbone fixed, its output for a given image never changes, so you compute it **once** and fit the head on the cached features. That is not an optimisation, it is a consequence of freezing.

What this still cannot do

- **Say where the flower is.** The model gives a label for a whole image and no location at all
- **Handle more than one object.** Two species in one photograph is a question this architecture cannot express
- **Say how confident it is, defensibly.** Softmax outputs are not calibrated probabilities, as Lecture 4 showed on a much simpler model

The first two are the subject of Lecture 14, and they need a different notion of what a correct answer is — which is why that lecture opens by deriving one.

If you keep one line from today

THE LINE

Do not learn what somebody has already learned for you. 1,020 images cannot pay for 4.8 million parameters, and they do not have to: the bottom of the stack is about photographs, not about flowers, and it transfers.

The measured version of that sentence is the ablation table: from 15.3% to well past it, on the same corpus, the same split and the same evaluation, by not training most of the network.

What we did

1. Named the condition that made Lecture 12 fail: 1,020 images paying for 4.8 million parameters, most of which encode general facts about photographs
2. Reused a backbone trained on a much larger corpus, and measured how far down the stack those features stay useful
3. Froze it and fitted a linear head; then unfroze the last block with a smaller learning rate for the parts that were already good
4. Met the batch-norm trap: running statistics keep updating even in a frozen block unless you stop them
5. Added augmentation, and stated what each transform *asserts* about the task — a flipped flower is a flower, a flipped digit is not
6. Ablated all three, and named the conditions under which transfer fails outright

NOT PRESENTED — FOR AFTERWARDS

Exercises

Lecture 13

five, in the style of the written paper

Exercises — Lecture 13

- 1** State the order in which a pretrained network's head must be replaced and its body frozen, and what goes wrong in the other order. [4]
- 2** Fine-tuning uses 10^{-4} on the pretrained block and 10^{-3} on the new head. Justify the difference in one sentence. [4]
- 3** Freezing a batch-norm layer's weights does not freeze the layer. State what still changes, and the call that stops it. [4]

Solutions on Lecture 14's deck.

Exercises — Lecture 13 (continued)

- 4** An augmented validation loader is used to select the epoch to keep. State the two things this costs, and the one-line test that detects it. [5]
- 5** A frozen-backbone probe is both more accurate and faster than training from scratch on 1,020 images. Explain why both, and name the cost that must still be counted. [5]

Solutions on Lecture 14's deck.

THE FIVE SET AT THE END OF LECTURE 12

Solutions Lecture 12

not part of this lecture

Solutions — Lecture 12

1. A dense layer mapping a $3 \times 128 \times 128$ input to a $32 \times 128 \times 128$ output has...

✓ $32 \times 3 \times 7 \times 7 = 4,704$.

Neither H nor W appears.

- One kernel per output channel, reused at every position
- The image size is genuinely absent — that is what weight sharing means

2. State the difference between equivariance and invariance, and say which of the two segmentation cannot afford.

✓ **Equivariance:** $f(Tx) = Tf(x)$.

Invariance: $f(Tx) = f(x)$.

Segmentation cannot afford invariance.

- Invariance discards the position, and the answer to a segmentation question *is* the position
- Classification wants invariance for the same reason

Solutions — Lecture 12 (2)

3. Convolution is equivariant to translation on the interior of an image but not at the border. Give the reason.

✓ Zero padding invents input that was not there, so a shift moves real content into invented content.

- The identity assumes the operator sees the same neighbourhood before and after the shift
- At the edge it does not, so the equality is an interior statement

4. A training run fails with out-of-memory. State which of parameters and activations usually dominates, and...

✓ Activations dominate; halve the batch.

- Activation memory is linear in the batch size and parameter memory does not depend on it at all
- Reducing the parameter count is near the bottom of the list

Solutions — Lecture 12 (3)

5. Parameters and activations are anti-correlated across a convolutional stack. State where each is...

**✓ Activations are largest near the input, parameters near the output.
Dense intuition points the wrong way.**

- Early layers have large maps and tiny kernels; late layers the reverse
- In a dense network both grow together, so the habit transfers badly

LECTURE 14 · GÉRON CH. 12

Detection and segmentation

IoU and mAP, derived — and why IoU gives no gradient when boxes are disjoint

The maximum that repairs the non-monotonicity of Lecture 3

Non-maximum suppression, semantic and instance segmentation

Run the Lecture 13 notebook first